First,we had discussed about all properties of Binary search tree.

Search in a Binary search Tree :-

https://leetcode.com/problems/search-in-a-binary-search-tree/

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {

        TreeNode* curr = root;

        while(curr!=NULL && curr->val!=key){

            if(key>curr->val)
                curr=curr->right;

            else
                curr= curr->left;
        }
        return curr;
    }
};
```

Insert in a Binary search Tree :-

```
class Solution {
public:
    // Function to insert a new node with given value in BST
    Node* insertNode(Node* root, int key) {
        // If tree is empty, create new node and return
        if (root == nullptr)
            return new Node(key);

        // If key is less, insert in left subtree
        if (key < root->val)
            root->left = insertNode(root->left, key);
        // If key is greater, insert in right subtree
```

```
        else
            root->right = insertNode(root->right, key);

        // Return the (unchanged) root node
        return root;
    }
};
```

Kth smallest in a BST :-

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode structure
// for a binary tree node
struct TreeNode {
    // Value of the node
    int val;

    // Pointer to the left child node
    TreeNode* left;

    // Pointer to the right child node
    TreeNode* right;

    // Constructor to initialize the node with a
    // value and set left and right pointers to null
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
private:
    // Helper function to perform inorder
    // traversal to find Kth smallest element
    void inorder(TreeNode* node, int& counter, int k, int& kSmallest) {
        if (!node || counter >= k) return;

        // Traverse left subtree
        inorder(node->left, counter, k, kSmallest);

        // Increment counter after visiting left subtree
        counter++;
```

```
        // Check if current node is the Kth smallest
        if (counter == k) {
            kSmallest = node->val;
            return;
        }

        // Traverse right subtree if
        // Kth smallest is not found yet
        inorder(node->right, counter, k, kSmallest);
    }

public:
    pair<int, int> findKth(TreeNode* root, int k) {
        int kSmallest = INT_MIN;
        // Counter to track visited nodes
        int counter = 0;

        // Find Kth smallest element
        // (perform inorder traversal)
        inorder(root, counter, k, kSmallest);


        return make_pair(kSmallest, kLargest);
    }
};
```

Floor in BST :-

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode structure
// for a binary tree node
struct TreeNode {
    // Value of the node
    int val;

    // Pointer to the left child node
    TreeNode* left;

    // Pointer to the right child node
    TreeNode* right;

    // Constructor to initialize the node with a
```

```
    // value and set left and right pointers to null
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Function to find the floor of a key
    // in a Binary Search Tree (BST)
    int floorInBST(TreeNode* root, int key){
        // Initialize the floor variable
        int floor = -1;

        // Traverse the BST until root becomes null
        while(root){
            // If key is equal to current node value
            // then it is the floor
            if(root->val == key){
                floor = root->val;
                return floor;
            }

            // If key is greater than current node value
            // move right and update floor
            if(key > root->val){
                floor = root->val;
                root = root->right;
            }
            else{
                // If key is smaller, move left
                root = root->left;
            }
        }
        // Return the computed floor value
        return floor;
    }
};

// Function to perform an in-order traversal
// of a binary tree and print its nodes
void printInOrder(TreeNode* root) {
    // Base case: if node is null
    if (root == nullptr) return;

    // Left subtree
    printInOrder(root->left);
```

```
    // Print current node
    cout << root->val << " ";

    // Right subtree
    printInOrder(root->right);
}

int main() {
    // Creating a BST
    TreeNode* root = new TreeNode(10);
    root->left = new TreeNode(5);
    root->right = new TreeNode(13);
    root->left->left = new TreeNode(3);
    root->left->left->left = new TreeNode(2);
    root->left->left->right = new TreeNode(4);
    root->left->right = new TreeNode(6);
    root->left->right->right = new TreeNode(9);
    root->right->left = new TreeNode(11);
    root->right->right = new TreeNode(14);

    cout << "Binary Search Tree (Inorder): "<< endl;
    printInOrder(root);
    cout << endl;

    Solution solution;

    // Key to find floor
    int target = 8;
    int floorVal = solution.floorInBST(root, target);

    if(floorVal != -1){
        cout << "Floor of " << target << " is: " << floorVal << endl;
    }
    else{
        cout << "No floor found!" << endl;
    }

    return 0;
}
```